

# Task 08 - Part 01: Question Answers

C# Fundamentals - Interfaces, Abstract Classes, IComparable & Encapsulation

## ***Q1. Why is it better to code against an interface rather than a concrete class?***

Coding against an interface (e.g., using an `IVehicle` reference instead of a `Car` reference) decouples the calling code from any specific implementation. This means the same code can work with any class that implements the interface - `Car`, `Bike`, or a future class not yet written - without modification, which makes the system more flexible and extensible. It also makes unit testing much easier (you can substitute a mock or fake implementation of the interface), and it enforces a clean separation between "what an object can do" (the contract) and "how it does it" (the implementation), which is a core principle of good object-oriented design (the Dependency Inversion Principle).

## ***Q2. When should you prefer an abstract class over an interface?***

An abstract class is the better choice when related classes need to **share common implementation code or state** (fields, concrete methods) in addition to a contract - not just a set of method signatures. For example, if `Rectangle` and `Circle` both need identical logic in a `Display()` method, putting that shared code in an abstract `Shape` base class avoids duplicating it in every derived class, which an interface cannot do (aside from default interface methods with no state). An abstract class also makes sense when there's a genuine "is-a" relationship and a natural single hierarchy, whereas an interface is better when unrelated classes need to share only a capability/contract, or when a class already has a base class and needs to add extra behavior (since C# allows multiple interfaces but only one base class).

## ***Q3. How does implementing IComparable improve flexibility in sorting?***

Implementing `IComparable<T>` (via the `CompareTo()` method) lets a custom class define its own "natural" ordering - for example, comparing `Product` objects by `Price`. Once that's in place, built-in .NET sorting functionality such as `Array.Sort()` or `List<T>.Sort()` can sort a collection of that type **automatically**, without needing any custom sorting algorithm or a separate comparison function written for every collection of that type. It also means the type integrates cleanly with any other .NET API that expects comparable objects (like sorted collections), making the sorting logic reusable and consistent everywhere the type is used, rather than duplicated at every call site.

## ***Q4. What is the primary purpose of a copy constructor in C#?***

The primary purpose of a copy constructor is to create a **new, independent object** whose fields are initialized from an existing object of the same type, giving you a true (deep) copy rather than just another reference to the same instance. This matters especially for reference types, where a simple assignment (`Student s2 = s1;`) only copies the reference - both variables still point to the same underlying object, so changing one affects the other. A properly written copy constructor instead duplicates the actual data (and, for a deep copy, duplicates any nested reference-type fields too), ensuring the copy can be modified safely without any side effects on the original object.

### ***Q5. How does explicit interface implementation help in resolving naming conflicts?***

Explicit interface implementation (writing `void IWalkable.Walk()` instead of a normal public method) lets a class provide a member that satisfies an interface's contract **without exposing it as part of the class's own public API**, and without a naming collision if the class needs a different, unrelated member with the same name for its own purposes. The explicitly implemented method can then only be called through a reference typed as the interface (e.g., `((IWalkable)robot).Walk()`), not through the class's own type. This is especially useful when a class implements two different interfaces that happen to declare a method with the identical name and signature, or when a method makes sense as an interface member but shouldn't clutter the class's normal public surface.

### ***Q6. What is the key difference between encapsulation in structs and classes?***

Encapsulation works largely the same way in both - private fields exposed through public properties or methods - but the underlying difference is that a struct's data is a **value type**, so encapsulating it protects the data within each individual copy of the struct, while a class's data is a **reference type**, so encapsulating it protects the single shared object that every reference points to. In practical terms, if a struct's internal state is exposed and copied around, each copy is fully independent, so "protecting" it mostly guards against invalid values being set on that particular instance. With a class, encapsulation additionally protects the object's state from being corrupted by any of the potentially many references pointing to that same shared instance.

### ***Q7. What is abstraction as a guideline, and what's its relation with encapsulation?***

**Abstraction** is the principle of exposing only the essential, relevant behavior of something while hiding the unnecessary implementation details - it's about designing a simplified, conceptual "what it does" view (like an `IVehicle` contract with `StartEngine()/StopEngine()`) rather than exposing "how it does it" internally. **Encapsulation** is the mechanism that makes abstraction possible in code: bundling data and behavior together and restricting direct access to internal state (via access modifiers, properties, etc.) is exactly what allows the messy implementation details to be hidden behind a clean interface. In short, abstraction is the design goal ("expose only what matters"), and encapsulation is the technical tool used to achieve it ("hide the rest").

### ***Q8. How do default interface implementations affect backward compatibility in C#?***

Before C# 8.0, adding a new member to an existing interface would break **every** class that already implemented that interface, since each of those classes would now fail to compile until it implemented the new member too. Default interface implementations solve this: a new method can be added to an interface **with a default body**, so existing implementing classes continue to compile and work exactly as before, automatically inheriting the default behavior, without being forced to change. Any class that does want different behavior can still explicitly override the default implementation. This makes it far safer to evolve and extend public interfaces (such as those in a shared library or NuGet package) over time without breaking every consumer of that interface.

***Q9. How does constructor overloading improve class usability?***

Constructor overloading lets a class offer several different ways to create an instance, depending on how much information the caller has available at that point - for example, creating a Book with just a Title, or with both Title and Author. This makes the class more convenient and flexible to use, since callers aren't forced to supply values they don't have yet or don't care about, and it avoids requiring extra setup code (like calling several property setters after construction) just to create a valid object in a common scenario. It keeps the class's public API expressive and self-documenting, since each constructor signature communicates a clear, valid way to build the object.